

曾涛



Github : <https://github.com/Orlandoo24>

☎ 13750545017 ✉ 0x70jiaru@gmail.com 🧑 籍贯广东 🎓 23 年 7 月本科毕业

涉及区块链钱包开发、游戏内容数据中心服务端开发、agent 开发等领域

热衷于通过技术改变生活，大二暑假开始实习参与企业级开发，毕业后在区块链技术团队担任核心开发

基本信息

全日制统招本科（2019.09 - 2023.07），广东白云学院 - 计算机科学与技术（CST - 英文国际班）

▪ 英文水平 CET4：502；广州白云区大学生软件设计比赛二等奖；计协副会长、校园极客工作室负责人

工作经历

（2024.05-至今）**腾讯** - 腾娱互动 - IEG 游戏流量生态部 - **Golang 后台 & 数开** - 武汉腾讯研发中心

职责：策略平台的数据拉取和数据计算流程的重构，实时监控和异步计算、数据匹配机制从0到1的设计；将策略数据出库时间缩短64%，并通过异步平缓写的机制降低了上游数据入库高峰期 ck 查询**57%的时延**和彻底杜绝了 readtimeout 事故；负责调研本部门的所有爬虫需求的开发可行性，并将方案给到爬虫数据供应商

（2022.08-2024.03）Astrosea - web3 部门 - **区块链 Java&Golang** - 杭州

职责：服务端订单引擎、支付引擎、虚拟资产分发链路；EVM、Btc、Solana 主流链的游戏内嵌多链热钱包、MPC 私钥管理、基于websocket 的交易订阅监听系统0到1的架构设计和开发，并且优化降低了监控逻辑中扫描最新区块链上数据和地址表比对过程中52%的数据库负载。

项目经验

腾讯IEG - 游可赢-游戏营销策略平台 - <https://yky.qq.com/>

▪ 技术栈 **Golang + Gin + Redis + Kafka + Clickhouse StarRocks + MySQL +**

项目背景：负责游戏营销策略服务的实时与离线数据链路建设。离线数据通过 Python 脚本+爬虫拉取清洗后直接入库，单时间写入量大的走 MQ；实时画像数据通过 Webhook 订阅进入 MQ，由 Go 消费端异步批量写入 CK / SR，并维护复杂 SQL 查询，为各游戏项目组提供精准的用户画像与投放策略分析。

项目难点：① 数据量逐渐增大，原部分 ck 在读写混合高峰下性能退化（RT ≥100ms、频繁 readtimeout）② 大投场景下上游数据激增容易引起 kafka 的积压和 ck 性能抖动频发，也容易引发内存风险

③ Kafka 消息在「At-least-once」语义下可能重复消费，原先用 CK 库表本身的机制去重导致增加了额外的负载

解决方案：①引入 Kafka 在消费端编写可靠保守的策略异步控制写入速率和批次 ② 并进行包括了3轮优化去保证数据可靠性和避免了 io 密集场景下 OOM、内存泄漏、goroutine 数量过大等隐患 ③ 供应商不做幂等的数据源通过通过 redis 存储 3h 窗口内的数据唯一 id 进行快速去重

方案成果：

① 彻底解决了大量读写混合场景下接口的查询性能退化问题；Redis 去重机制有效减少重复 insert 约 28%，降低 Merge 触发频率，降低数仓集群 63% 的 cpu 负载 ② 异步处理方案使系统吞吐量提升，能灵活支持多端下游去消费同一份数据 ③ MQ堆积和下游数仓宕机或者负载高等极端场景能有完善的方案兜底避免数据丢失 ④ 重构后洞玄支持多下游模块（实时画像、审计、BI）共享同一数据流，实现高可靠、高扩展的实时游戏、电商策略平台

小红书分布式社媒自动化 Agent 系统 (基于 MCP 协议) - <https://github.com/Orlandoo24/auto-content>

■ 技术栈 Go + Gin + Rod + Redis + Kafka + MCP + Docker + Kubernetes

项目背景：设计并开发了一个基于 MCP 协议的高性能自动化代理系统，支持通过 LLM 自然语言指令控制社媒账号（小红书）进行数据爬取分析和内容裂变发布。

核心贡献 & 难点解决：

- 架构解耦：将单体应用重构为 Master-Worker 分布式架构。使用 Queue 实现任务异步分发，支持 Worker 节点基于 Kubernetes 的弹性伸缩，单集群支持并发 200+ 浏览器实例运行。
- 性能优化：针对浏览器启动耗时高的问题，设计了 Browser Context Pool (浏览器上下文池)。通过复用底层 CDP 连接，将单次任务平均响应时间从 2.5s 降低至 0.8s，CPU 资源利用率提升 40%。
- 分布式状态管理：废弃本地文件存储，基于 Redis 构建了集中式 Session 管理中心。实现了多账号的 Cookies 自动续期与 分布式锁 机制，确保同一账号操作的原子性，有效规避平台风控。
- 标准化接口：实现了 Model Context Protocol (MCP) 标准，使系统能够无缝接入 Claude/OpenAI 等大模型生态，支持 10+ 种复杂业务场景的 Function Calling。

技术清单

- Golang
 - 能够熟练使用 Golang 进行服务端开发，熟悉 Golang 的基本语言特性
- 工具
 - 熟悉如何编写 MCP 服务、使 agent 能调用外部服务，熟悉 skills 的编写；能使得 agent 有更强的 code review 能力和问题排查能力，熟悉并深度体验过各类 ai 辅助编程软件，企业版 curosr、trae、claude-code，内网版腾讯 codebuddy
- Python
 - 能够熟练用爬虫组件 **pypeteer** 编写爬虫脚本，fast-api 框架构建图文转视频 AI 服务
- Java
 - Java 是启蒙语言，曾参与构建 SpringCloud + Dubbo + Nacos 构建的微服务架构流媒体中台的开发，能用以集成 Kafka 完成游戏事件回传的日志收集和事件流处理。
- 数据库：
 - 熟悉 **MySQL**，了解 InnoDB 的基本特性好引擎下的索引的使用、数据结构
 - 熟悉分析型数据库 **StarRocks**，了解如何使用其构建交易历史日志数仓
- Redis：熟悉 **Redis** 各种场景下的缓存、分布式锁的应用
- MQ
 - 熟悉 Kafka 基础架构、网络通信模型等，具有事务消息实战设计经验；能用 Kafka 对大批量日志、流媒体数据、爬虫等处理需要一定时间的数据进行异步处理